

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Вычислительная математика»
Тема: Метод Ньютона

Студент гр. 4342

Митюков М.Н.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы

Используя программы-функции NEWTON и Round из файла methods.cpp, найти корень уравнения $f(x) = 0$ с заданной точностью Eps методом Ньютона, а также исследовать скорость сходимости и чувствительность метода к ошибкам округления.

Задание

В случае, когда известно хорошее начальное приближение решения уравнения $f(x) = 0$, эффективным методом повышения точности является метод Ньютона. Он состоит в построении итерационной последовательности $x_{n+1} = x_n - f(x_n)/f'(x_n)$, сходящейся к корню уравнения $f(x) = 0$. Достаточные условия сходимости метода формулируются теоремой, приведенной в [1,7].

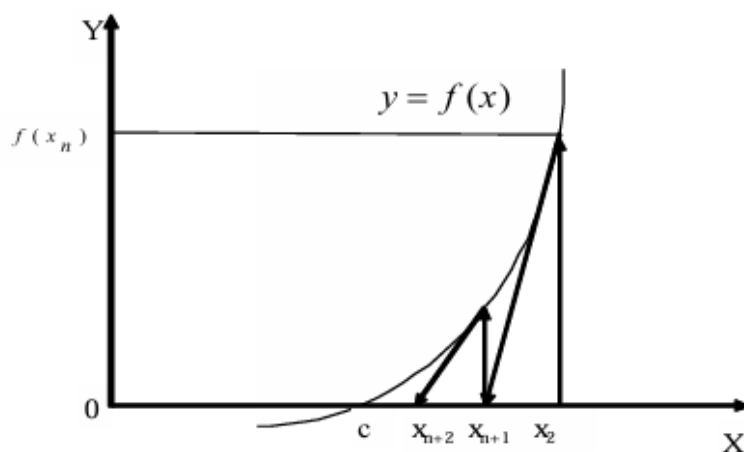


Рис.2

Метод Ньютона допускает простую геометрическую интерпретацию (рис. 2). Если через точку с координатами $(x_n; f(x_n))$ провести касательную, то абсцисса точки пересечения этой касательной с осью OX будет очередным приближением x_{n+1} корня уравнения $f(x) = 0$. Для оценки погрешности n-го приближения корня предлагается пользоваться неравенством

$|c - x_n| \leq \frac{M_2}{2m_1} |x_n - x_{n-1}|^2$, где M_2 — наибольшее значение модуля второй производной $|f''(x)|$ на отрезке $[a, b]$; m_1 — наименьшее значение модуля первой производной $|f'(x)|$ на отрезке $[a, b]$. Таким образом, если $|x_n - x_{n-1}| < \varepsilon$, то $|c - x_n| \leq \frac{M_2 \varepsilon}{2m_1}$. Это означает, что при хорошем начальном приближении корня после каждой итерации число верных десятичных знаков в очередном приближении удваивается, т.е. процесс сходится очень быстро (имеет место

квадратическая сходимость). Из указанного следует, что при необходимости нахождения корня с точностью итерационный процесс можно прекращать, когда

$$|x_n - x_{n-1}| < \varepsilon = \sqrt{2m_1\varepsilon/M_2}. \quad (3.1)$$

Рассмотрим один шаг итераций. Если на $(n-1)$ -м шаге очередное приближение x_{n-1} не удовлетворяет условию окончания процесса, то вычисляются величины $f(x_{n-1}), f'(x_{n-1})$ и следующие приближение корня $x_n = x_{n-1} - f(x_{n-1})/f'(x_{n-1})$. При выполнении условия (3.1) величина x_n принимается за приближенное значение корня α , вычисленное с точностью ε . В лабораторной работе № 5 предлагается, используя программы-функции NEWTON и ROUND из файла methods.cpp (файл заголовков methods.h, директория LIBR1), найти корень уравнения $f(x) = 0$ с заданной точностью Eps методом Ньютона, исследовать скорость сходимости и обусловленность метода. Для данной работы вид функции $f(x)$ задается индивидуально каждому студенту преподавателем из числа вариантов, приведенных в подразделе 3.6.

Порядок выполнения лабораторной работы №5.

- 1) Графически или аналитически отделить корень уравнения $f(x) = 0$ (т.е. найти отрезки $[Left, Right]$, на котором функция $f(x)$ удовлетворяет условиям сходимости метода Ньютона).
- 2) Составить подпрограммы - функции вычисления $f(x), f'(x)$, предусмотрев округление их значений с заданной точностью Delta.
- 3) Составить головную программу, вычисляющую корень уравнения $f(x) = 0$ содержащую обращение к подпрограммам $f(x), f'(x)$, Round, NEWTON и индикацию результатов.
- 4) Выбрать начальное приближение корня x_0 из $[Left, Right]$ так, чтобы $f(x) f''(x) > 0$

- 5) Провести вычисления по программе. Исследовать скорость сходимости метода и чувствительность метода к ошибкам в исходных данных.

Для приближенного вычисления корней уравнения $f(x) = 0$ методом Ньютона предназначена программа - функция NEWTON, текст которой представлен в подразделе 3.7.

Выполнение работы

1. Отделение корня и выбор начального приближения

Рассматривается функция $f(x) = \sin(x^2) - 6x + 1$, с областью определения $\mathbb{R}$. На выбранном интервале $[0; 0.5]$ функция непрерывна. Значения на концах интервала имеют разные знаки:

1. При $x = 0$:

$$f(0) = \sin(0) - 6 \cdot 0 + 1 = 1 > 0.$$

2. При $x = 0.5$:

$$f(0.5) = \sin(0.5^2) - 6 \cdot 0.5 + 1 = \sin(0.25) - 3 \approx 0.247 - 2 \approx -1.753 < 0.$$

Так как $f(0) > 0$ и $f(0.5) < 0$, то: $f(0) \cdot f(0.5) < 0$.

Следовательно, на отрезке $[0; 0.5]$ существует корень уравнения $f(x) = 0$. Для метода Ньютона дополнительно проверяются производные функции. В работе используются выражения

$$f'(x) = 2x \cdot \cos(x^2) - 6,$$

$$f''(x) = 2\cos(x) - 2x \cdot \sin(x).$$

На выбранном промежутке $f'(x) < 0$ и $f''(x) > 0$. Поэтому функция убывает и является выпуклой вверх. В качестве начального приближения выбрано $x_0 = 0$, так как выполняется условие сходимости $f(x_0) \cdot f''(x_0) > 0$.

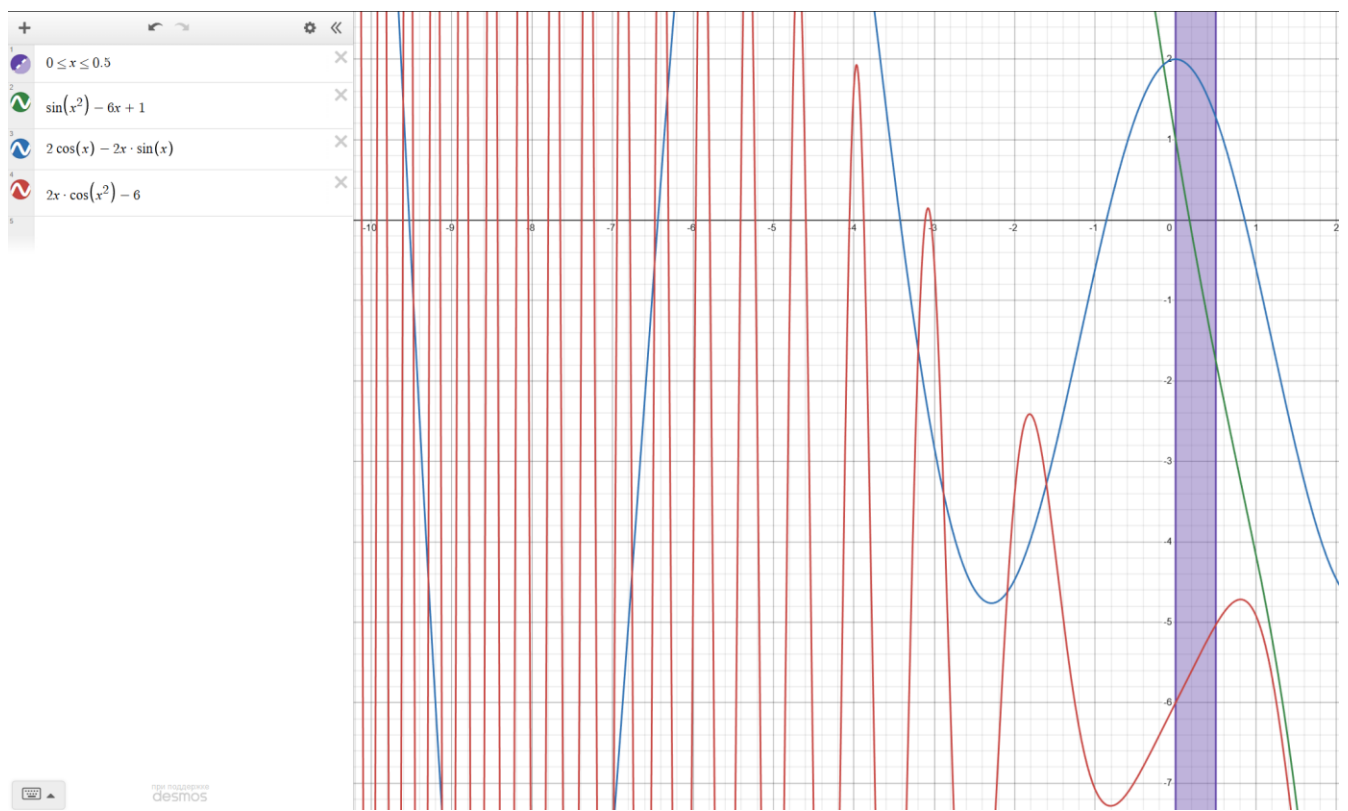


Рисунок 2 — График функции и производных на отрезке $[0;0.5]$

2. Теоретические оценки для критерия остановки

В реализации используется оценка погрешности метода Ньютона. На отрезке $[0; 0.5]$ берутся константы

$$m_1 = \min |f'(x)| = |f'(0.5)| \approx 5.03109,$$

$$M_2 = \max |f''(x)| = |f''(0)| = 2.$$

Итерационный процесс прекращается, когда разность двух соседних приближений становится меньше величины

$$|x_n - x_{n-1}| < \varepsilon_0 = \sqrt{(2m_1 \text{Eps} / M_2)}.$$

Такой критерий соответствует теоретической оценке погрешности и используется в функции NEWTON напрямую.

3. Описание программной реализации

В файле main.cpp заданы функции $f(x)$, $f'(x)$ и $f''(x)$.

В файле methods.cpp реализованы:

- функция Round(X, Delta), округляющая число до ближайшего значения, кратного Delta;
- функция SetRoundDelta(Delta), задающая используемую точность округления;
- функция SetNewtonConstants(m₁, M₂), передающая в метод Ньютона значения m₁ и M₂;
- функция NEWTON(X, Eps, N), выполняющая итерации $x_{n+1} = x_n - f(x_n)/f'(x_n)$.

На каждом шаге метода сначала вычисляются $f(x_n)$ и $f'(x_n)$, затем при необходимости они округляются с помощью Round. После этого находится следующее приближение. Если производная обращается в нуль, точность Eps задана неверно, либо очередное приближение выходит из области определения, программа аварийно завершается с сообщением об ошибке.

4. Результаты исследования сходимости

Сначала был найден опорный корень при Eps = 10⁻¹² и Delta = 0:

$$x_{\text{ref}} \approx 0.171572123742971, \quad N = 4.$$

Далее метод Ньютона запускался для разных значений Eps без округления функции и производной. Результаты приведены в таблице 1.

Eps	Число итераций N	Найденный корень
1e-1	1	0.166666666666667
1e-2	1	0.166666666666667
1e-3	2	0.171567885856705
1e-4	2	0.171567885856705
1e-5	2	0.171567885856705
1e-6	3	0.171572123739804
1e-8	3	0.171572123739804

Eps	Число итераций N	Найденный корень
1e-10	3	0.171572123739804
1e-12	4	0.171572123742971

Таблица 1 — Зависимость числа итераций метода Ньютона от точности Eps

По таблице видно, что метод Ньютона сходится очень быстро: для Eps от 10^{-1} до 10^{-2} хватает одной итерации, для Eps от 10^{-3} до 10^{-5} требуется две итерации, а для более строгой точности — три. Это подтверждает высокую скорость сходимости метода при удачно выбранном начальном приближении.

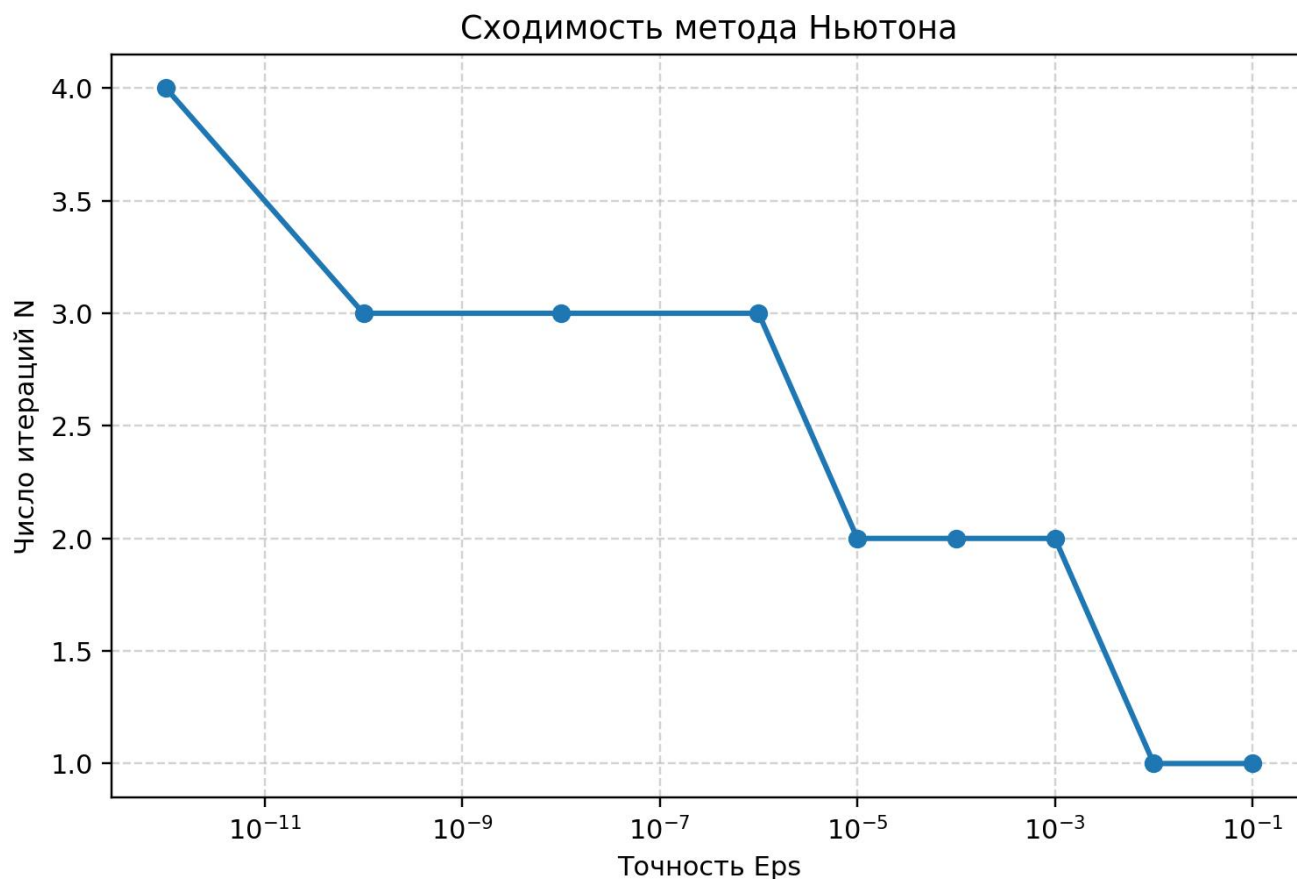


Рисунок 3 — Зависимость числа итераций от точности Eps

5. Исследование чувствительности к ошибкам округления

При фиксированной точности $Eps = 10^{-8}$ параметр Delta изменялся от 0 до 10^{-1} . Округление применялось к значениям $f(x)$ и $f'(x)$ на каждом шаге метода. Абсолютная ошибка вычислялась как $|x - x_{ref}|$. Результаты сведены в таблицу 2.

Delta	N	Найденный корень	$ x - x_{ref} $
0	3	0.171572123742971	0.0000000000
1e-1	2	0.166666666666667	0.004905457076305
1e-2	3	0.171957671957672	0.000385548214700
1e-3	3	0.171607552496912	0.000035428753940
1e-4	3	0.171572433589798	0.000000309846826
1e-5	3	0.171572442751562	0.000000319008591
1e-6	3	0.171572092308559	0.000000031434413

Таблица 2 — Влияние параметра Delta на результат метода Ньютона

По мере уменьшения Delta найденный корень стабилизируется и приближается к опорному значению. Это означает, что метод Ньютона чувствителен к грубому округлению, но при малых значениях Delta сохраняет высокую точность.



Рисунок 4 — Зависимость абсолютной ошибки от точности округления Δ

Вывод

В лабораторной работе реализован метод Ньютона для решения уравнения $\sin(x^2) - 6x + 1 = 0$. Аналитически и графически показано, что корень находится на отрезке $[0; 0.5]$, а условие $f(x_0) \cdot f''(x_0) > 0$ позволяет выбрать начальное приближение $x_0 = 0$. Реализация метода показала квадратичную скорость сходимости: при разумном начальном приближении для достижения высокой точности достаточно 2–3 итераций. Исследование чувствительности показало, что метод резко теряет точность при грубом округлении значений функции и производной, но при малом Δ остаётся устойчивым.

ПРИЛОЖЕНИЕ А

Название файла `main.cpp`

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include "methods.h"

//  $f(x) = \sin(x^2) - 6x + 1$ 
// Интервал изоляции:  $[0; 0.5]$ 

double F(double x)
{
    return  $\sin(x*x) - 6*x + 1$ ;
}

double DF(double x)
{
    return  $2*x*\cos(x*x) - 6$ ;
}

double DDF(double x)
{
    return  $2*\cos(x) - 2*x*\sin(x)$ ;
}

int main()
{
    double Left = 0.0;
    double Right = 0.5;
    // Начальное приближение выбираем из условия  $f(x_0) * f'(x_0) > 0$ .
    double X0 = 0.0;
    double RefRoot, Root;
    double Eps, Delta;
    double m1, M2;
    int N;
```

```

printf("f(x) = sin(x^2) - 6x + 1\n");

printf("Interval otdeleniya kornya: [%.2f; %.2f]\n", Left, Right);
printf("F(Left)    = %.10f\n", F(Left));
printf("F(Right)   = %.10f\n", F(Right));
printf("DF(Left)   = %.10f\n", DF(Left));
printf("DF(Right)  = %.10f\n", DF(Right));
printf("DDF(Left)  = %.10f\n", DDF(Left));
printf("DDF(Right)= %.10f\n\n", DDF(Right));

// Оценки min |f'(x)| и max |f''(x)| на отрезке [0, 0.5].
m1 = fabs(DF(Right));
M2 = fabs(DDF(Left));
SetNewtonConstants(m1, M2);
printf("m1 = min |f'(x)| = %.10f\n", m1);
printf("M2 = max |f''(x)| = %.10f\n", M2);
printf("x0 = %.10f\n\n", X0);

// Нахождение опорного корня с высокой точностью
SetRoundDelta(0.0);
RefRoot = NEWTON(X0, 1e-12, N);
printf("(Eps=1e-12, Delta=0): %.15f (Iterations: %d)\n\n",
RefRoot, N);

// Исследование скорости сходимости
printf("Eps\t\tIterations\tRoot\n");
double EpsValues[] = {1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-6, 1e-8,
1e-10, 1e-12};
int nEps = sizeof(EpsValues) / sizeof(EpsValues[0]);

FILE* feps = fopen("newton_iterations_vs_eps.csv", "w");
fprintf(feps, "Eps,Iterations,Root\n");

for (int i = 0; i < nEps; i++) {
    Eps = EpsValues[i];
    SetRoundDelta(0.0);

```

```

        Root = NEWTON(X0, Eps, N);
        printf("%.1e\t%d\t\t%.15f\n", Eps, N, Root);
        fprintf(feps, "%e,%d,%.15f\n", Eps, N, Root);
    }
    fclose(feps);

    // Исследование чувствительности к ошибкам округления
    printf("\n(Eps=1e-8):\n");
    printf("Delta\t\tIterations\tRoot\t\t\tError\n");

    double DeltaValues[] = {0.0, 1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-6};
    int nDelta = sizeof(DeltaValues) / sizeof(DeltaValues[0]);

    FILE* fdel = fopen("newton_sensitivity_vs_delta.csv", "w");
    fprintf(fdel, "Delta,Iterations,Root,Error\n");

    for (int i = 0; i < nDelta; i++) {
        Delta = DeltaValues[i];
        SetRoundDelta(Delta);
        Root = NEWTON(X0, 1e-8, N);
        double Err = fabs(Root - RefRoot);
        printf("%.1e\t%d\t\t%.15f\t%.15f\n", Delta, N, Root, Err);
        fprintf(fdel, "%e,%d,%.15f,%.15f\n", Delta, N, Root, Err);
    }
    fclose(fdel);

    return 0;
}

```

Название файла **methods.h**

```

#ifndef METHODS_H
#define METHODS_H

#include <cmath>

// Внешние функции варианта задаются в main.cpp.
extern double F(double x);

```

```

extern double DF(double x);
extern double DDF(double x);

// Установка точности округления значений функции и производной.
// Если Delta = 0, округление не выполняется.
void SetRoundDelta(double Delta);

// Функция Round - округление числа X до ближайшего значения, кратного
Delta.
double Round(double X, double Delta);

// Установка констант для критерия остановки метода Ньютона.
void SetNewtonConstants(double m1, double M2);

// Метод Ньютона для поиска корня уравнения  $F(x) = 0$ .
// X    - начальное приближение корня
// Eps  - требуемая погрешность вычисления корня
// N    - число выполненных итераций
// Округление значений функции и производной задается через SetRoundDelta.
// Критерий остановки:  $|x_n - x_{(n-1)}| < \sqrt{2*m1*Eps/M2}$ 
double NEWTON(double X, double Eps, int &N);

#endif

```

Название файла methods.cpp

```

#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include "methods.h"

static double gDelta = 0.0;
static double gM1 = 0.0;
static double gM2 = 0.0;

```



```

// Установка шага округления значений функции и производной.
void SetRoundDelta(double Delta)
{
    gDelta = Delta;
}

// Установка констант m1 и M2 для критерия остановки.
void SetNewtonConstants(double m1, double M2)
{
    gM1 = m1;
    gM2 = M2;
}

// =====
// Функция Round - округление числа до заданной точности Delta
// =====
double Round(double X, double Delta)
{
    if (Delta <= 0.0) return X;
    if (Delta < 1E-12) {
        printf("Неверное задание точности округления\n");
        exit(1);
    }

    if (X >= 0.0)
        return Delta * floor(X / Delta + 0.5);
    else
        return Delta * ceil(X / Delta - 0.5);
}

// =====
// Метод Ньютона
//  $x_{n+1} = x_n - f(x_n) / f'(x_n)$ 
// Критерий остановки согласно оценке погрешности:
//  $|x_n - x_{n-1}| < \sqrt{2 \cdot m1 \cdot \text{Eps} / M2}$ 
// =====

```

```

double NEWTON(double X, double Eps, int &N)
{
    // Проверка корректности входных данных.
    if (Eps <= 0.0) {
        printf("Неверное задание точности\n");
        exit(1);
    }
    if (gM1 <= 0.0 || gM2 <= 0.0) {
        printf("Не заданы константы m1 и M2 для метода Ньютона\n");
        exit(1);
    }

    // Порог остановки по априорной оценке погрешности метода Ньютона.
    double E0 = sqrt(2.0 * gM1 * fabs(Eps) / gM2);
    double XPrev = X;
    double FX, DFX, XNext;

    N = 0;

    // Основной итерационный процесс метода Ньютона.
    while (true) {
        FX = Round(F(XPrev), gDelta);
        DFX = Round(DF(XPrev), gDelta);

        // Производная в текущей точке не должна обращаться в нуль.
        if (DFX == 0.0) {
            printf("Производная равна нулю. Метод Ньютона неприменим.\n");
            exit(1);
        }

        XNext = XPrev - FX / DFX;
        N++;

        // Критерий остановки по разности соседних приближений.
        if (fabs(XNext - XPrev) < E0)
            return XNext;
    }
}

```

```

// Новое приближение должно оставаться в области определения.
if (XNext <= 0.0) {
    printf("Следующее приближение вышло из области определения\n");
    exit(1);
}

// Защита от заикливания при нарушении сходимости.
if (N > 1000) {
    printf("Превышено допустимое число итераций\n");
    exit(1);
}

XPrev = XNext;
}
}

```

Название файла: plot.py

```

import csv
import math
import matplotlib.pyplot as plt
import matplotlib.path as pe

# График 1: число итераций от точности Eps
rows = []
with open('newton_iterations_vs_eps.csv', encoding='utf-8') as fcsv:
    reader = csv.DictReader(fcsv)
    for row in reader:
        rows.append((float(row['Eps']), int(row['Iterations'])))
rows.sort(key=lambda t: t[0])
eps = [r[0] for r in rows]
iters = [r[1] for r in rows]

plt.figure(figsize=(8.0, 5.0))
plt.semilogx(eps, iters, marker='o', linewidth=2.0)
plt.xlabel('Точность Eps')
plt.ylabel('Число итераций N')
plt.title('Сходимость метода Ньютона')
plt.grid(True, which='both', linestyle='--', alpha=0.55)
plt.savefig('newton_iterations_v18.png', dpi=220, bbox_inches='tight')
plt.close()

# График 2: погрешность корня от точности округления Delta
rows = []

```

```

with open('newton_sensitivity_vs_delta.csv', encoding='utf-8') as fcsv:
    reader = csv.DictReader(fcsv)
    for row in reader:
        d = float(row['Delta'])
        if d > 0.0:
            rows.append((d, float(row['Error'])))
rows.sort(key=lambda t: t[0])
delta = [r[0] for r in rows]
err = [r[1] for r in rows]

plt.figure(figsize=(8.0, 5.0))
plt.loglog(delta, err, marker='o', linewidth=2.0)
plt.xlabel('Точность округления Delta')
plt.ylabel('|x - x_ref|')
plt.title('Чувствительность метода Ньютона к округлению')
plt.grid(True, which='both', linestyle='--', alpha=0.55)
plt.savefig('newton_sensitivity_v18.png', dpi=220, bbox_inches='tight')
plt.close()

```